



Energy learning hyper-heuristic algorithm for cooperative task assignment of heterogeneous UAVs under complex constraints

Mengshun Yuan, Mou Chen^{*}, Tongle Zhou, Zengliang Han

College of Automation Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing, 211106, China

ARTICLE INFO

Article history:

Received 18 March 2025

Received in revised form

11 May 2025

Accepted 4 June 2025

Available online 12 June 2025

Keywords:

Unmanned aerial vehicle

Cooperative task assignment

Energy learning

Hyper-heuristic algorithm

ABSTRACT

Cooperative task assignment is one of the key research focuses in the field of unmanned aerial vehicles (UAVs). In this paper, an energy learning hyper-heuristic (EL-HH) algorithm is proposed to address the cooperative task assignment problem of heterogeneous UAVs under complex constraints. First, a mathematical model is designed to define the scenario, complex constraints, and objective function of the problem. Then, the scheme encoding, the EL-HH strategy, multiple optimization operators, and the task sequence and time adjustment strategies are designed in the EL-HH algorithm. The scheme encoding is designed with three layers: task sequence, UAV sequence, and waiting time. The EL-HH strategy applies an energy learning method to adaptively adjust the energies of operators, thereby facilitating the selection and application of operators. Multiple optimization operators can update schemes in different ways, enabling the algorithm to fully explore the solution space. Afterward, the task order and time adjustment strategies are designed to adjust task order and insert waiting time. Through the iterative optimization process, a satisfactory assignment scheme is ultimately produced. Finally, simulation and experiment verify the effectiveness of the proposed algorithm.

© 2025 China Ordnance Society. Publishing services by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The rapid advancements in unmanned aerial vehicle (UAV) technology have revolutionized modern warfare, enabling a wide range of operations that were previously difficult or impossible to conduct [1,2]. The UAVs have the characteristics of flexibility, mobility, and autonomy [3–5]. These characteristics make UAVs indispensable for various military tasks such as reconnaissance, electronic jamming, communication relay, strike, and cargo transport [6–8]. In battlefield scenarios, the complexity and unpredictability of operational environments necessitate the use of multiple UAVs working in concert to achieve task objectives [9,10]. In the cooperative operation of multiple UAVs, cooperative task assignment is required [11]. Effective task assignment ensures optimal assignment and scheduling of tasks, maximizing task success while adhering to complex operational constraints [12]. Over the past years, numerous algorithms have been proposed to address the cooperative task assignment problem [13–15].

Cooperative task assignment algorithms can be broadly categorized into five primary methods: exact methods, heuristic methods, meta-heuristic methods, learning-based methods, and hyper-heuristic methods [16,17]. Exact methods, such as integer linear programming (ILP) [18] and mixed-integer linear programming (MILP) [19], guarantee optimal solutions by exhaustively exploring all possible assignments [20]. These methods offer the highest level of accuracy but incur high computational costs, requiring substantial time and resources. Heuristic methods, including greedy algorithm [21] and rule-based algorithm [22], offer fast and straightforward solutions. They work by making locally optimal choices at each step with the hope of finding global optimal solution [23]. These methods are easy to implement but prone to getting trapped in local optimal solution. Meta-heuristic methods, such as genetic algorithm (GA) [24], particle swarm optimization algorithm (PSO) [25], grey wolf optimizer algorithm (GWO) [26], artificial bee colony algorithm (ABC) [27], and whale optimization algorithm (WOA) [28], provide more robust solutions by exploring a larger search space. Meta-heuristic algorithms are inspired by natural processes and behaviors, enabling them to find high-quality solutions [29]. These algorithms offer a good balance between solution quality and computational efficiency, making

^{*} Corresponding author.

E-mail address: chenmou@nuaa.edu.cn (M. Chen).

Peer review under the responsibility of China Ordnance Society.

them suitable for simple task assignment problems. Learning-based methods, such as reinforcement learning (RL) [30] and deep learning (DL) [31], leverage historical data and environmental feedback to improve UAV target assignments over time. Learning-based methods can adapt to dynamic environments and learn complex patterns, offering high flexibility and the potential for near-optimal solutions [32]. However, they require extensive training data and computational resources, and exhibit poor stability in practical applications. Hyper-heuristic algorithms represent the higher-level algorithms by managing a set of low-level heuristic operators or meta-heuristic operators [33]. In Ref. [34], the multitask-oriented manufacturing service composition (MMSC) model was effectively addressed by the proposed genetic-based hyper-heuristic (GA-HH) algorithm. In Ref. [35], the PSO-based hyper-heuristic algorithm was proposed to solve the multi-robot task allocation problem. In Ref. [36], the learning-based selection hyper-heuristic framework (LS-HH) was presented to effectively optimize the distributed heterogeneous hybrid blocking flow-shop scheduling problem. In Ref. [37], the researchers proposed a clustering-based hyper-heuristic algorithm for coverage path planning of heterogeneous UAVs. Hyper-heuristic algorithms aim to automate the heuristic selection process, providing a flexible and adaptive solution framework that improves the efficiency and quality of task assignment. For different optimization problems, it is necessary to design appropriate hyper-heuristic algorithms to solve them. The cooperative task assignment problem of heterogeneous UAVs under complex constraints is currently a difficult non-convex composite optimization problem due to differences in UAV payloads and numerous constraints. The above methods encounter issues such as easily getting trapped in local optimal solution, unsuitable solution encoding, and slow convergence when addressing this problem.

Based on the above literature analysis, this paper proposes an energy learning hyper-heuristic (EL-HH) algorithm to address the cooperative task assignment problem of heterogeneous UAVs under complex constraints. The EL-HH algorithm is validated according to simulation and experiment, demonstrating its effectiveness in improving task success and resource utilization. The main contributions are as follows:

- 1) A mathematical model for the cooperative task assignment problem under complex constraints is designed. Based on this model, a comprehensive objective function is presented, including positive rewards and negative penalties.
- 2) In the EL-HH algorithm, a three-layer encoding scheme is proposed, including task sequence, UAV sequence, and waiting time. And, the EL-HH strategy is introduced. This strategy effectively guides the selection and application of optimization operators, ensuring that the optimization process remains dynamic and adaptive. Meanwhile, multiple optimization operators are designed and implemented. These operators include both selection and action operators, which play crucial roles in generating schemes.
- 3) The task order and time adjustment strategies are presented based on the directed graph to adjust the task sequence and execution time. By adjusting task order and inserting waiting time, the strategies ensure compliance with the task order constraints while striving to meet the time window constraints.

The organization of this paper is given as follows. Section 2 introduces the problem formulation. The EL-HH algorithm is proposed in Section 3. Section 4 showcases the simulation and experiment. Finally, Section 5 concludes this paper.

2. Problem formulation

2.1. Cooperative task assignment scenario

The cooperative task assignment problem involves coordinating multiple UAVs to execute a set of tasks efficiently and effectively, as shown in Fig. 1. In a battlefield cooperative operation environment, multiple UAVs carry different payloads and are assigned different tasks. Multiple UAVs are heterogeneous due to different payloads. This problem requires addressing complex constraints and objectives, such as maximizing task completion, minimizing total flight time, and adhering to payload capacities. In combat scenarios, multiple UAVs are assigned to perform reconnaissance, electronic jamming, electronic deception, communication relay, and strike tasks. Due to the differences in UAV payloads and the numerous constraints, the cooperative task assignment problem is a difficult non-convex composite optimization problem, requiring advanced algorithms to find efficient and feasible solutions. The known information regarding the cooperative task assignment problem can be represented as [27]:

$$\Gamma = \{\mathcal{O}, T, C, U\} \quad (1)$$

with

$$\begin{cases} \mathcal{O} = \{Map, Therats\} \\ T = \{T_1, T_2, \dots, T_{N_T}\} \\ C = \{C_1, C_2, \dots, C_{N_T}\} \\ U = \{U_1, U_2, \dots, U_{N_U}\} \end{cases} \quad (2)$$

where $\mathcal{O}$ represents the set of obstacles and threats, *Map* represents the obstacle map, *Therats* represents the set of threats, T represents the set of tasks, N_T is the number of tasks, C represents the set of task values, U represents the set of UAVs, and N_U is the number of UAVs. In the following, T_i represents the i -th task and U_j represents the j -th UAV.

2.2. Complex constraints

- 1) Task conflict constraints: Each task T_i needs to be executed by exactly one UAV. The constraint can be expressed as [38]:

$$\sum_{j=1}^{N_U} s_{ij} = 1, \quad \forall i \in \{1, 2, \dots, N_T\} \quad (3)$$

where s_{ij} is a boolean value indicating whether UAV U_j is assigned to task T_i .

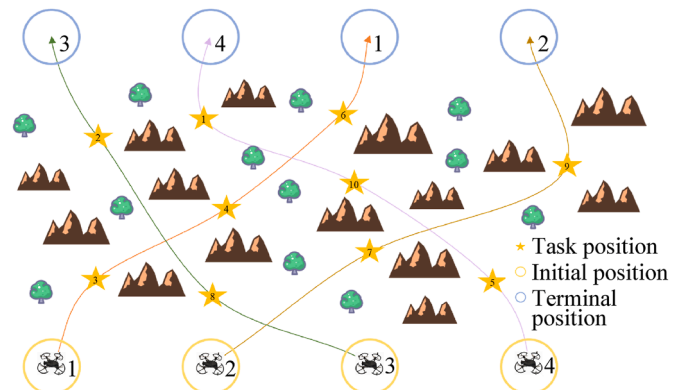


Fig. 1. The cooperative task assignment problem of multiple UAVs.

2) Task type constraints: The tasks include reconnaissance T_{Rec} , electronic jamming T_{Jam} , electronic deception T_{Dec} , communication relay T_{Com} , and strike T_{Str} . Each task type has unique requirements and impacts the overall task cost. The set of task types $\mathcal{T}$ can be expressed as:

$$\mathcal{T} = \{T_{\text{Rec}}, T_{\text{Jam}}, T_{\text{Dec}}, T_{\text{Com}}, T_{\text{Str}}\} \quad (4)$$

where $\mathcal{T}_i$ represents the i -th task type.

3) Task duration constraints: A task T_i has a fixed duration D_i [22]. Let $\text{Start}(T_i)$ denote the actual start time of task T_i , and $\text{End}(T_i)$ denote the actual end time of task T_i , then $D_i = \text{End}(T_i) - \text{Start}(T_i)$.

4) Task order constraints: There may be order constraints between different tasks. If task T_i must start before task T_j , this constraint is expressed as:

$$\text{Start}(T_i) \leq \text{Start}(T_j) \quad (5)$$

5) Task time window constraints: A task may have time window constraint, denoted as $[E_{s,i}, E_{c,i}]$, where $E_{s,i}$ represents the earliest start time and $E_{c,i}$ represents the latest end time.

6) Task position constraints: Tasks must be performed at specific positions [25]. The position of task T_i is denoted as $\text{Pos}(T_i)$. The position of UAV U_j is denoted as $\text{Pos}(U_j)$. The $\text{Dis}(\text{Pos}(U_j), \text{Pos}(T_i))$ represents the path cost from the current position of UAV U_j to the position of task T_i .

7) Heterogeneous UAV payload constraints: A UAV can be equipped with various payloads, including electro-optical pods L_{Rec} , electronic jamming equipment L_{Jam} , electronic deception equipment L_{Dec} , communication relay payloads L_{Com} , and strike missiles L_{Str} . Each UAV can carry one or more of these payloads. The set of payload types $\mathcal{L}$ can be expressed as:

$$\mathcal{L} = \{L_{\text{Rec}}, L_{\text{Jam}}, L_{\text{Dec}}, L_{\text{Com}}, L_{\text{Str}}\} \quad (6)$$

where $\mathcal{L}_i$ represents the i -th payload type. The payload type corresponds to the task type, and payload type $\mathcal{L}_i$ can complete task type $\mathcal{T}_i$.

8) UAV initial and terminal position constraints: The initial position $P_{\text{ini},j}$ and terminal position $P_{\text{ter},j}$ of UAV U_j are fixed [10].

9) UAV safety constraints: The UAVs must avoid obstacles on the map. The Route_j represents the flight path of UAV U_j . The paths of UAVs must avoid all obstacles, which can be expressed as:

$$\text{Route}_j \cap \mathcal{O} = \emptyset, j = 1, 2, \dots, N_U \quad (7)$$

10) UAV missile load constraints: When a UAV carries multiple missiles, there is an upper limit on the payload capacity. The missile load capacity of UAV U_j is denoted as B_j . The missile load constraint of UAV U_j is expressed as:

$$\sum_{i=1}^{N_T} M_{ij} S_{ij} \leq B_j \quad (8)$$

where M_{ij} represents the number of missiles required for task T_i by UAV U_j .

11) UAV flight time constraints: The UAV U_j flies at a constant speed v_j , with a maximum flight time R_j . The task set of UAV U_j is denoted as $J_j = \{J_{j,1}, J_{j,2}, \dots, J_{j,N_j}\}$, where N_j represents the

number of tasks. The total flight time F_j of UAV U_j is expressed as:

$$F_j = \sum_{k=1}^{N_j} \left(\frac{\text{Dis}(\text{Pos}(U_j), \text{Pos}(J_{j,k}))}{v_j} + W_{J_{j,k}} + D_{J_{j,k}} \right) + \frac{\text{Dis}(\text{Pos}(U_j), P_{\text{ter},j})}{v_j} \quad (9)$$

where $W_{J_{j,k}}$ is the waiting time of task $J_{j,k}$, $D_{J_{j,k}}$ is the duration of task $J_{j,k}$.

The flight time constraint of UAV U_j is expressed as $F_j \leq R_j$.

2.3. Objective function

The objective function aims to maximize task completion while minimizing costs, including positive rewards and negative penalties.

1) Task completion reward: The objective function rewards UAVs for completing assigned tasks. The task completion reward function is expressed as:

$$J_{\text{task}} = \sum_{i=1}^{N_T} \sum_{j=1}^{N_U} C_i S_{ij} W_{ij} \quad (10)$$

where C_i represents the value of task T_i , and the binary number w_{ij} indicates whether the payload of UAV U_j can complete task T_i .

2) Energy consumption reward: The objective function rewards UAVs for minimizing total flight time. The energy consumption reward function is expressed as:

$$J_{\text{energy}} = \sum_{j=1}^{N_U} \max(0, (R_j - F_j)) \quad (11)$$

3) Missile load constraint penalty: If a UAV exceeds its missile load capacity, it incurs a penalty proportional to the excess load. The missile load penalty function is expressed as:

$$J_{\text{load}} = \sum_{j=1}^{N_U} \max\left(0, \sum_{i=1}^{N_T} M_{ij} S_{ij} - B_j\right) \quad (12)$$

4) Flight time constraint penalty: If a UAV exceeds its maximum flight time, it incurs a penalty proportional to the excess time. The flight time penalty function is expressed as:

$$J_{\text{time}} = \sum_{j=1}^{N_U} \max(0, F_j - R_j) \quad (13)$$

5) Time window constraint penalty: If the task execution time does not satisfy the task time window constraint, a penalty is applied. The time window penalty function is expressed as:

$$J_{\text{window}} = \sum_{i=1}^{N_T} (\max(0, E_{s,i} - \text{Start}(T_i)) + \max(0, \text{End}(T_i) - E_{c,i})) \quad (14)$$

The overall objective function combines the positive rewards and negative penalties to optimize the cooperative task

assignment. The weighted sum of rewards and penalties $J_{\text{sum}}(X)$ is expressed as:

$$J_{\text{sum}}(X) = \alpha_1 J_{\text{task}} + \alpha_2 J_{\text{energy}} - \alpha_3 J_{\text{load}} - \alpha_4 J_{\text{time}} - \alpha_5 J_{\text{window}} \quad (15)$$

where X is the scheme, $\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5$ are positive coefficients corresponding to each term.

To facilitate the optimization and presentation, the final objective function $J(X)$ is designed as:

$$J(X) = \frac{1}{2} \left(\frac{2}{\pi} \arctan \left(\frac{J_{\text{sum}}(X)}{\beta} \right) + 1 \right) \quad (16)$$

where β is a positive coefficient, and $J(X) \in (0, 1)$. The final objective function has both upper and lower bounds, which facilitates accurate energy adjustment based on cost changes for the subsequent EL-HH strategy.

3. Cooperative task assignment using the EL-HH algorithm

3.1. The framework of the EL-HH algorithm

The composition diagram of the EL-HH algorithm is shown in Fig. 2. Firstly, the scheme encoding is designed in three layers: the first layer represents the task sequence, the second layer represents the UAV sequence, and the third layer represents the waiting time before the tasks are executed. Then, the EL-HH strategy is proposed. This strategy updates the energy and probability of each operator by evaluating the fitness of the scheme. Based on these probability distributions, the strategy selects appropriate selection and action operators to produce the task-UAV sequence. Subsequently, multiple groups of operators are designed to select and update schemes. Furthermore, task order and time adjustment strategies are designed to eliminate task order and time window constraints. Through these designs, the EL-HH algorithm iteratively optimizes and ultimately produces a satisfactory assignment scheme.

3.2. Three-layer encoding scheme

To effectively address the cooperative task assignment problem, a three-layer encoding scheme is designed, as shown in Fig. 3. The scheme consists of three layers: the task sequence, the UAV sequence, and the task waiting time. Based on this encoding scheme, the schemes generated by the EL-HH algorithm can satisfy all constraints.

1) First layer: Task sequence

Each task needs to be ordered in a sequence. In task sequence, the order of tasks assigned to the same UAV can be directly determined. For tasks assigned to multiple UAVs, their sequence needs to be determined based on the actual start time. A scheme at this layer can be expressed as:

$$X_{\text{tasks}} = [T_3, T_1, T_4, T_2, \dots, T_{N_T}] \quad (17)$$

If T_3 and T_1 are assigned to the same UAV in the sequence, it indicates that the UAV will first execute T_3 , followed by T_1 .

2) Second layer: UAV sequence

This layer determines which UAV is assigned to each task in the task sequence from the second layer. A scheme at this layer can be expressed as:

$$X_{\text{UAVs}} = [U_2, U_1, U_3, U_2, \dots, U_{N_U}] \quad (18)$$

This sequence indicates that the first task in the task sequence is assigned to U_2 , the second task to U_1 , and so on.

3) Third layer: Waiting time

The third layer encodes the wait time of all tasks. This layer determines the waiting time required before a UAV can execute a task. A scheme at this layer can be expressed as:

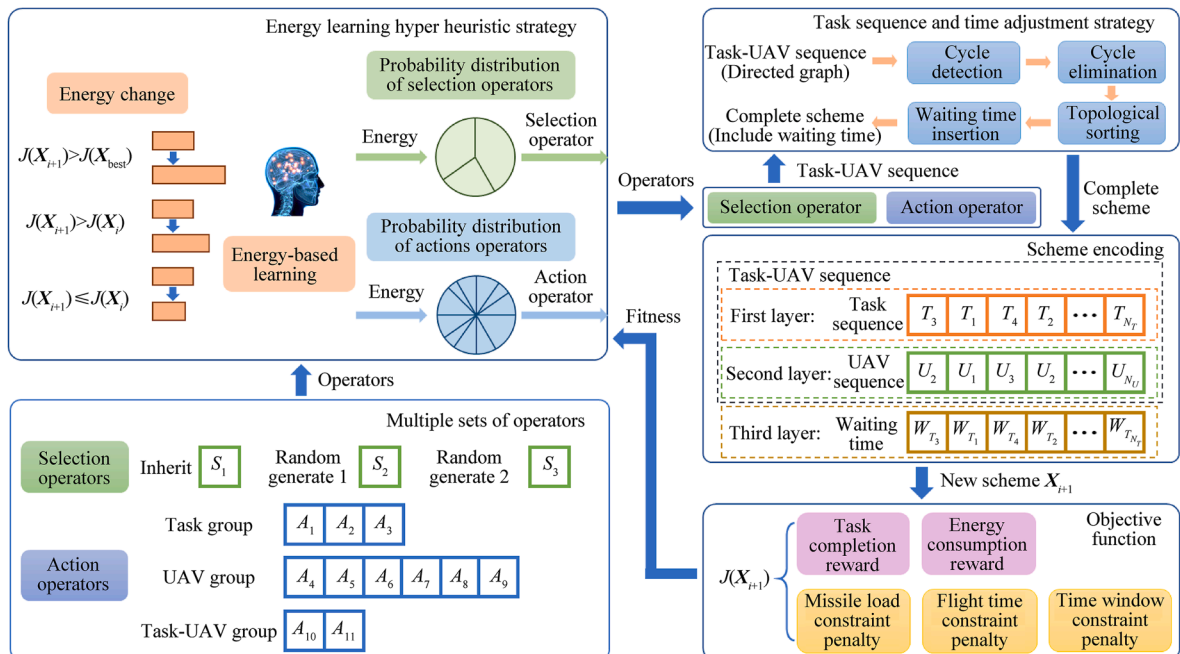


Fig. 2. The composition diagram of the EL-HH algorithm.

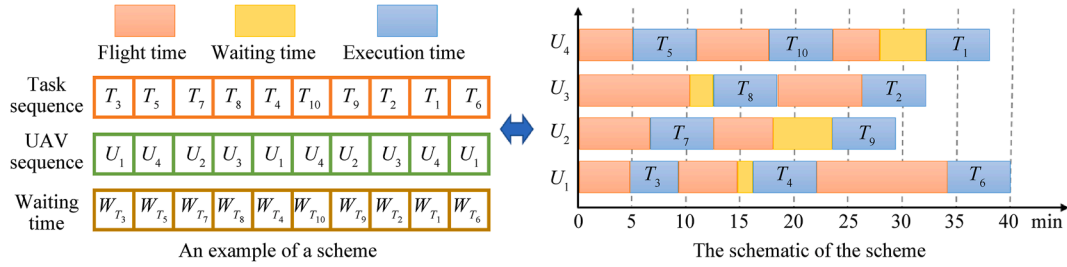


Fig. 3. The three-layer encoding scheme.

$$W_{\text{time}} = [W_{T_3}, W_{T_1}, W_{T_4}, W_{T_2}, \dots, W_{T_{N_T}}] \quad (19)$$

This encoding indicates that the waiting time of first task T_3 is W_{T_3} , the waiting time of second task T_1 is W_{T_1} , and so on.

The complete scheme is encoded through these three layers. As shown in Fig. 3, a complete scheme is expressed as:

t , and γ is a positive constant. Accordingly, the expression for the cumulative probability $Q_k(t)$ of the k -type operator is expressed as:

$$Q_k(t) = \sum_{l=1}^k P_k(l). \quad (23)$$

$$X = [X_{\text{tasks}}, X_{\text{UAVs}}, W_{\text{time}}] = [[T_3, T_1, T_4, T_2, \dots, T_{N_T}], [U_2, U_1, U_3, U_2, \dots, U_{N_U}], [W_{T_3}, W_{T_1}, W_{T_4}, W_{T_2}, \dots, W_{T_{N_T}}]] \quad (20)$$

The task-UAV sequence is expressed as:

$$Y = [X_{\text{tasks}}, X_{\text{UAVs}}] = [[T_3, T_1, T_4, T_2, \dots, T_{N_T}], [U_2, U_1, U_3, U_2, \dots, U_{N_U}]] \quad (21)$$

In this example, the task sequence $[T_3, T_1, T_4, T_2, \dots, T_{N_T}]$ is paired with the UAV sequence $[U_2, U_1, U_3, U_2, \dots, U_{N_U}]$, indicating that U_2 will execute T_3 , U_1 will execute T_1 , and so on. By employing this three-layer encoding scheme, the EL-HH algorithm can effectively search the search space.

3.3. The EL-HH strategy

The EL-HH strategy learns and adjusts the probability distribution of different operators based on the energy learning. During the iterative process, operators that improve the solution will have their corresponding energy increased, which in turn raises their selection probability; otherwise, their energy and selection probability will decrease. In each generation, this strategy selects the selection and action operators based on their respective probability distributions. These operators are applied to generate the task-UAV sequence. After inserting the waiting time, a complete scheme is formed. The fitness of the scheme is then calculated. Based on the fitness, the energies and probabilities of the operators are updated for the next iteration. Taking the action operators as an example, the details of this strategy are introduced. Assume there are M types of action operators, each with energy E_k and selection probability P_k . The $P_k(t)$ represents the selection probability of k -type operator in generation t . The $P_k(t)$ follows Boltzmann probability distribution, which can be expressed as [39]:

$$P_k(t) = \frac{\exp(\gamma E_k(t))}{\sum_{l=1}^M \exp(\gamma E_l(t))} \quad (22)$$

where $E_k(t)$ represents the energy of k -type operator in generation

Then, an energy function correlated with the operators is proposed, where the function decays as the number of iterations progresses. And the energy function increases upon instances where the operator enhances the fitness value. The function is further constrained by predefined lower bound $E_{\min} > 0$ and upper bound $E_{\max} > 0$. The energy update expression of k -type operator $E_k(t)$ is expressed as:

$$E_k(t+1) = \max \left(E_{\min}, \min \left(E_{\max}, E_k(t) \cdot e^{-\alpha(t-t_{f,k})} + \sum_{q=1}^{N_{k,t}} \Delta E_q \right) \right) \quad (24)$$

where $E_k(t+1)$ is the energy of k -type operator in next iteration $t+1$, α is a positive constant that determines the rate of decay, $e^{-\alpha(t-t_{f,k})}$ represents the exponential decay term, $N_{k,t}$ is the number of k -type operators in current generation t . If the operator enhances the fitness in generation t , the algorithm sets $t_{f,k}$ to the current generation, $t_{f,k} = t$. The ΔE_q represents the change value of energy when the q -th k -type operator changes the fitness value.

The calculation formula of ΔE_q is expressed as:

$$\Delta E_q = \begin{cases} E_o \left(a + \frac{(1-a)t}{T_{\max}} \right) & \text{if } J(X_i^{t+1}) > J(X_{\text{best}}) \\ E_h \left(a + \frac{(1-a)t}{T_{\max}} \right) & \text{elseif } J(X_i^{t+1}) > J(X_i^t) \\ E_r & \text{else} \end{cases} \quad (25)$$

where $a \in [0.5, 1]$ is a constant, $T_{\max}$ is the maximum number of iterations, E_o, E_h, E_r are the energy change values, and $E_o > E_h > 0 > E_r$. Besides, X_i^t is the corresponding scheme of the operator, which represents the i -th scheme in the t -th generation.

This strategy enables the EL-HH algorithm to rapidly search around the current best scheme and avoid getting trapped in local optimal solution. These advantages allow the EL-HH algorithm to efficiently explore the solution space.

3.4. Low-level operators

Low-level operators include selection operators and action operators.

3.4.1. Selection operators

Selection operators determine how base task-UAV sequences are obtained.

S_1 : Inherit task-UAV sequence of the same serial number from the previous generation.

$$Y_i^{t+1} = Y_i^t \quad (26)$$

where Y_i^t represents the i -th task-UAV sequence in the t -th generation.

S_2 : Randomly generate task-UAV sequence by adhering to UAV payload constraints.

$$Y_i^{t+1} = [\pi_T([T_1, T_2, \dots, T_{N_T}]), [U_{\pi_U(1)}, U_{\pi_U(2)}, \dots, U_{\pi_U(N_U)}]] \quad (27)$$

where the random permutation function $\pi_T(\cdot)$ can generate the random sequence of tasks. There are no duplicate tasks in the task sequence generated by the random permutation function. The $\pi_U(i)$ randomly selects a UAV number from $\{1, 2, \dots, N_U\}$, ensuring that the UAV's payload meets the requirements of the corresponding task.

S_3 : Randomly generate task-UAV sequence while adhering to the task order and UAV payload constraints.

$$Y_i^{t+1} = [\text{Seq}(\pi_T([T_1, T_2, \dots, T_{N_T}])), [U_{\pi_U(1)}, U_{\pi_U(2)}, \dots, U_{\pi_U(N_U)}]] \quad (28)$$

where the function $\text{Seq}(\cdot)$ adjusts the task sequence so that it meets the task order constraints regardless of UAV sequence.

Remark 1. The function $\text{Seq}(\cdot)$ iteratively swaps the positions of two tasks that violate the task order constraints to ensure that the task sequence complies with these constraints, without considering the UAV sequence. After calculating the actual execution time based on the UAV sequence, the complete scheme may still fail to satisfy the task order constraints.

3.4.2. Action operators

Action operators define the specific behaviors for updating the task-UAV sequence. These operators can be further divided by their operating groups, including the task group, the UAV group, and the task-UAV group. Operators in the task group act only on the first-level task sequence of the solution. Operators in the UAV group act only on the second-level UAV sequence. Operators in the task-UAV group operate on the task-UAV pairs within the scheme. The operators in the first two groups do not interfere with the other sequence, which might already be well-optimized. Similarly, the operators in the third group do not disrupt well-matched task-UAV pairs. This classification allows the algorithm to explore the solution space more effectively during the iterative process, thereby improving its efficiency.

1) Task group: The task group operators modify only the task sequence.

A_1 : Randomly swap two tasks.

A_2 : Randomly swap two adjacent tasks.

A_3 : Select a sequence of random position and length, and insert it into a random feasible position.

2) UAV group: The UAV group operators modify only the UAV sequence.

A_4 : Randomly mutate a UAV.

A_5 : Mutate a sequence of random position and length.

A_6 : Perform the single-point random crossover with another random scheme.

A_7 : Perform the single-point random crossover with the best scheme.

A_8 : Perform the multiple-point random crossover with another random scheme.

A_9 : Perform the multiple-point random crossover with the best scheme.

3) Task-UAV group: The task-UAV group operators modify the scheme by altering the pairs of tasks and UAVs.

A_{10} : Randomly select a task-UAV pair and insert it into the feasible random position.

A_{11} : Randomly select n task-UAV pairs and insert them into the feasible random position.

The operations of all action operators are shown in Fig. 4. These operators are designed to introduce diversity and guide the search process.

3.5. Task order and time adjustment strategies

3.5.1. Task order adjustment strategy

In the scheme, task order constraints between different tasks may lead to deadlock, where tasks mutually restrict one another and hinder execution. To solve this issue, the task order adjustment strategy based on directed graph is designed to identify and eliminate deadlocks. The task order adjustment strategy begins by constructing a directed graph $G = (V, E)$, where V represents task nodes and E represents edges. The edges E are formed by the task execution order in the scheme. Additionally, task order constraints are incorporated into E . If there is a loop in the directed graph, it indicates the presence of a deadlock in the scheme. The depth-first search (DFS) algorithm [40] is then used to detect the loop in the directed graph. Afterward, the two tasks in the loop that violate the task order constraints are selected and swapped. The pseudocode of the task order adjustment strategy is shown in Fig. 5.

3.5.2. Task time adjustment strategy

In the scheme, the actual execution time of tasks must meet both task order constraints and time window constraints. To address this issue, the task time adjustment strategy is designed to add the waiting time of tasks. For the directed graph G formed by the task-UAV sequence, the topological sort [41] is performed to obtain a new ordered task list T_{new} . If a task in T_{new} has no order and time window constraints, no waiting time is added. If the task's execution time does not satisfy the task order constraint, waiting time is added to ensure that the scheme meets the order constraint. The calculation formula for the waiting time W_{T_i} of T_i is expressed as:

$$W_{T_i} = \begin{cases} \text{Start}(T_b) - \text{Arrive}(T_i) & \text{if } \text{Arrive}(T_i) < \text{Start}(T_b) \\ 0 & \text{else} \end{cases} \quad (29)$$

where $\text{Arrive}(T_i)$ represents the time that the UAV arrives at the position of T_i , T_b is the task before T_i .

Then, if the task's execution time does not meet the time window constraint, waiting time is added to satisfy the time window constraint. The updated formula for the waiting time of T_i is expressed as:

$$W_{T_i} = \begin{cases} E_{s,i} - \text{Arrive}(T_i) & \text{if } \text{Arrive}(T_i) + W_{T_i} < E_{s,i} \\ \text{else} \end{cases} \quad (30)$$

The pseudocode of the time adjustment strategy is shown in

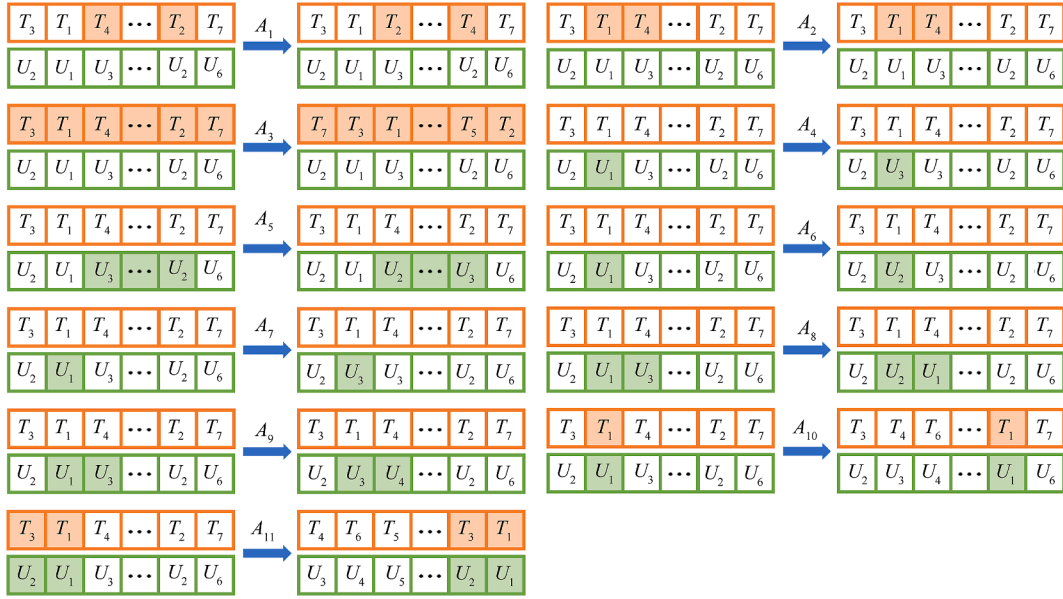


Fig. 4. The operation of all action operators.

Algorithm 1: Task order adjustment strategy**Input:** Task-UAV sequence**Output:** Task-UAV sequence without deadlock

- 1 Initialize directed graph $G = (V, E)$;
- 2 Use DFS algorithm to find loops lps in G ;
- 3 **while** $lps \neq \emptyset$ **do**
- 4 Select two tasks T_i and T_j in the loop that violate order constraint;
- 5 Swap the positions of the two tasks in the sequence;
- 6 Update G with the new order;
- 7 Use DFS algorithm to find loops lps in G ;

Fig. 5. The pseudocode of the task order adjustment strategy.

Algorithm 2: Task time adjustment strategy**Input:** Task-UAV sequence, directed graph $G = (V, E)$ **Output:** Complete scheme with waiting time

- 1 Perform topological sort on G to obtain a new task list T_{new} ;
- 2 **for** $T_i \in T_{new}$ **do**
- 3 **if** T_i does not satisfy the order constraint **then**
- 4 Calculate the waiting time using Eq. (29);
- 5 Add the waiting time to T_i to satisfy order constraint;
- 6 **if** T_i does not satisfy the time window constraint **then**
- 7 Calculate the waiting time using Eq. (30);
- 8 Add the waiting time to T_i to satisfy time window constraint as much as possible;

Fig. 6. The pseudocode of the time adjustment strategy.

Fig. 6. In the time adjustment strategy, the topological sort is applied to create an ordered list that respects the order constraints and the task-UAV sequence. The strategy iterates through each task, adding waiting time to ensure compliance with order constraints and to meet time window constraints as much as possible. Once all tasks are processed, the complete scheme with waiting time is returned.

Remark 2. The task order adjustment strategy and the task time adjustment strategy are interdependent. The task time adjustment strategy can only be executed after the task order adjustment strategy is completed. Otherwise, the task time adjustment will

fail due to the deadlocks in the scheme.

Remark 3. The task time adjustment strategy ensures that the actual flight scheme satisfies the task order constraints. However, this strategy only partially addresses the time window constraints. Specifically, if the task's arrival time exceeds a certain threshold relative to the task's earliest start time, this strategy cannot solve the time window constraint.

3.6. Pseudocode of the EL-HH algorithm

The initialization of the i -th scheme is randomly generated by adhering to UAV payload constraints, as shown in:

$$X_i^t = [\pi_T([T_1, T_2, \dots, T_{N_T}]), [U_{\pi_U(1)}, U_{\pi_U(2)}, \dots, U_{\pi_U(N_T)}], [0, 0, \dots, 0]] \quad (31)$$

In each iteration, a new scheme is compared with the previous generation, and the better one is retained. The retention rule is expressed as:

$$X_i^{t+1} = \begin{cases} X_i^{t+1} & \text{if } J(X_i^{t+1}) > J(X_i^t) \\ X_i^t & \text{else} \end{cases} \quad (32)$$

The pseudocode of the EL-HH algorithm is shown in Fig. 7. The inputs are maximum iterations T_{max} and population size M . The EL-HH algorithm starts by initializing all operators and schemes. Then, it uses a path planning algorithm to obtain feasible paths and distances between various positions, including task-to-task, task-to-UAV initial position, and task-to-UAV terminal position. When the paths and distances between various positions are known, there is no need for path planning. During each iteration, for each scheme, a selection operator is chosen to generate a base task-UAV sequence. An action operator is then selected to update the scheme. Afterward, the task order and time adjustment strategies are used to adjust task order and insert waiting time. After updating all schemes, the energy values of all operators are updated accordingly. The selection probabilities of the operators are also updated based on their new energy values. This iterative

Algorithm 3: Energy learning hyper-heuristic algorithm**Input:** Maximum iterations $T_{\max}$, population size M **Output:** Best scheme X_{best}

```

1 Initialize the energies and probabilities of all selection operators;
2 Initialize the energies and probabilities of all action operators;
3 Initialize all task-UAV sequences using Eq. (31) and set  $t = 1$ ;
4 Get feasible paths and costs between various positions;
5 Execute the task order and time adjustment strategy;
6 Form all complete schemes;
7 Calculate the fitness of all schemes using Eq. (16);
8 while  $t < T_{\max}$  do
9   for  $i = 1 : M$  do
10    Select the selection operator  $S$  using Eq. (23);
11    Get the task-UAV sequence  $Y_i^{t+1}$  through  $S$ ;
12    Select the action operator  $A$  using Eq. (23);
13    Update the task-UAV sequence  $Y_i^{t+1}$  through  $A$ ;
14    Execute the task order and time adjustment strategies;
15    Form the complete scheme  $X_i^{t+1}$ ;
16    Calculate the fitness of scheme using Eq. (16);
17    Record the fitness corresponding to  $S$  and  $A$  in  $i$ -th scheme;
18    Record the current best scheme  $X_{\text{best}}$ ;
19    Update the scheme  $X_i^{t+1}$  using Eq. (32);
20  Update the energies of all selection operators using Eq. (24);
21  Update the probabilities of all selection operators using Eq. (22);
22  Update the energies of all action operators using Eq. (24);
23  Update the probabilities of all action operators using Eq. (22);
24   $t = t + 1$ ;

```

Fig. 7. The pseudocode of the EL-HH algorithm.

process continues until the maximum number of iterations is reached, resulting in the best scheme X_{best} .

Remark 4. When task assignment requires rapidity and the paths are unknown, the EL-HH algorithm uses distance estimation to directly obtain the distance between various positions. After the task assignment is complete, the EL-HH algorithm plans the required path in the scheme.

3.7. Analysis of computational complexity

In task assignment algorithms, the computational complexity of the distance calculation and the adjust strategies can vary significantly depending on the specific situation. Therefore, in the analysis of the computational complexity of the EL-HH algorithm, the path planning component and adjust strategies are not considered. Then, the computational complexity of the EL-HH algorithm is primarily composed of the initialization process and the iterative optimization process. The total complexity is:

$$T_{\text{EL-HH}} = T_1 + T_0 = O(T_{\max} \cdot M \cdot N_X) \quad (33)$$

with

$$T_0 = O(T_{\max} \cdot M \cdot N_X) \quad (34)$$

where T_1 is the complexity of the initialization process, T_0 is the complexity of the iterative optimization process, N_X is the dimension of scheme, and $O(\cdot)$ is the asymptotic descriptor.

In solving the cooperative task assignment problem, it is essential to consider the algorithm's probabilistic completeness and convergence speed. Thanks to the inclusion of selection operators S_2 and S_3 , the algorithm can fully explore the solution space and avoid local optimal solution as the number of iterations increases. This ensures both probabilistic completeness and convergence to the global optimum. Furthermore, the EL-HH strategy and multiple groups of operators enable faster convergence of the algorithm. Overall, the EL-HH algorithm provides an efficient solution to the cooperative task assignment problem.

4. Simulation and experiment

4.1. Simulation

The simulation includes simple and complex scenes. The EL-HH algorithm is compared with the PSO, GWO, ABC, and WOA algorithms. To ensure the comparative fairness, the following measures are implemented:

- 1) The same paths and path costs are used for all task assignment algorithms.
- 2) All comparative algorithms are adjusted to accommodate the specific problem at hand. For instance, each algorithm only changes the task order when operating the task sequence.
- 3) The population initialization method and cost function of all algorithms are identical. All algorithms use the designed task order and time adjustment strategies in each iteration.
- 4) Each algorithm is run independently 20 times, with the optimal result taken as the final scheme. The cost iteration curves are averaged over these runs.

The key parameters of all algorithms are provided in Table 1.

4.1.1. Simple scene

A simple scene with a flight space of $20 \text{ km} \times 20 \text{ km} \times 5 \text{ km}$ is created. In this scene, 5 UAVs have to complete 10 tasks. The UAV information is presented in Table 2, and the task information is shown in Table 3. The maximum iterations $T_{\max}$ is limited to 500, and the population size M is defined as 50. When the EL-HH algorithm is run once, the cumulative probability change curves for selection operators are shown in Fig. 8(a), and the cumulative probability change curves for action operators are shown in Fig. 8(b). These figures demonstrate the variation in operator selection probabilities during the iterations. This process validates that the EL-HH strategy can efficiently adjust the selection probabilities of multiple low-level operators. It also confirms that the hyper-heuristic strategy can quickly focus the search around the current optimal scheme while effectively avoiding local optimal solution.

The optimal results achieved by all algorithms over multiple runs are listed in Table 4. The average cost variation curves over multiple runs are shown in Fig. 9. As illustrated in Fig. 9, the EL-HH algorithm achieves faster convergence speed in the optimization process. The multiple groups of optimization operators in the algorithm are beneficial for thoroughly exploring the solution space. Meanwhile, the EL-HH strategy can effectively select the suitable operators, thereby significantly accelerating the convergence speed. Fig. 10(a) provides a chronological view, highlighting the order and duration of tasks. The black lines in the figure indicate that the UAVs have reached their respective terminal positions. Fig. 10(b) visually maps out the flight paths, offering a clear representation within the map. In the figure, the orange squares

Table 1

The main parameters of all algorithms.

Algorithm	Main parameter	Value
PSO	Inertia weight	1
	Social factor	1.3
	Cognition factor	1.3
GWO	Initial distance control parameter	2
ABC	Predefined threshold limit	10
WOA	Initial convergence factor	2
EL-HH	Energy lower bound	1
	Energy upper bound	10

Table 2
UAV information in the simulation.

Scene	UAV	Initial position/km	Terminal position/km	Load type	Number of Missiles	Maximum time
Simple	U_1	(1, 18.5, 0.5)	(1, 18, 0.5)	L_{Rec}, L_{Dec}	0	70
	U_2	(1, 12.5, 0.5)	(1, 12, 0.5)	L_{Jam}, L_{Com}	0	40
	U_3	(1, 2, 0.6)	(1.5, 2, 0.6)	L_{Rec}, L_{Dec}	0	50
	U_4	(10, 1, 0.5)	(10.5, 1, 0.5)	L_{Jam}, L_{Str}	2	60
	U_5	(18, 1, 0.5)	(18.5, 1, 0.5)	L_{Com}, L_{Str}	1	70
Complex	U_1	(1, 10.5, 1.3)	(1, 10, 1.3)	L_{Rec}, L_{Dec}	0	40
	U_2	(1, 5.5, 0.6)	(1, 5, 0.6)	L_{Jam}, L_{Com}	0	65
	U_3	(1, 2, 0.3)	(1.5, 2, 0.3)	L_{Rec}, L_{Dec}	0	60
	U_4	(5, 1, 0.5)	(5.5, 1, 0.5)	L_{Jam}, L_{Str}	2	87
	U_5	(10, 1, 1.2)	(10.5, 1, 1.2)	L_{Com}, L_{Str}	2	70
	U_6	(13.5, 1, 1.6)	(14, 1, 1.6)	L_{Rec}, L_{Dec}	0	85

Table 3
Task information in the simulation.

Scene	UAV	Position/km	Task type	Prior Task	Missile quantity	Execution/min	Window/min	Value
Simple	T_1	(7, 19, 1.2)	T_{Rec}	/	0	8	[5,20]	5
	T_2	(11.5, 16.3, 1.2)	T_{Com}	/	0	12	/	4
	T_3	(6, 11, 1)	T_{Com}	/	0	12	/	4
	T_4	(10.8, 15.3, 1)	T_{Str}	/	1	4	[40, 55]	6
	T_5	(7, 7, 1)	T_{Dec}	/	0	10	/	1
	T_6	(10, 12, 1)	T_{Dec}	/	0	10	/	3
	T_7	(14.5, 8.5, 1)	T_{Str}	T_5	1	4	/	6
	T_8	(17.1, 15, 1.5)	T_{Jam}	/	0	8	/	7
	T_9	(17.5, 11, 1)	T_{Com}	/	0	10	[9,23]	4
	T_{10}	(19, 17, 1)	T_{Rec}	T_3	0	11	/	5
Complex	T_1	(2, 15, 1.3)	T_{Rec}	/	0	8	[5,15]	6
	T_2	(4, 18.5, 0.2)	T_{Dec}	T_4	0	12	/	2
	T_3	(4.5, 15.5, 1.4)	T_{Com}	T_1	0	11	[10,25]	4
	T_4	(6.5, 12.5, 0.9)	T_{Str}	/	1	4	/	3
	T_5	(6, 6, 0.7)	T_{Dec}	/	0	7	/	5
	T_6	(9.2, 10, 1)	T_{Dec}	/	0	6	[10,19]	1
	T_7	(9.6, 11.5, 1.3)	T_{Str}	/	1	6	/	7
	T_8	(12, 15, 1.6)	T_{Jam}	T_3	0	4	/	2
	T_9	(9.2, 16.8, 1.5)	T_{Com}	/	0	6	[20,35]	3
	T_{10}	(13.5, 12, 1.3)	T_{Rec}	/	0	13	/	5
	T_{11}	(15.3, 7, 1.8)	T_{Dec}	/	0	7	/	4
	T_{12}	(14.3, 9.2, 1.8)	T_{Dec}	/	0	8	/	6
	T_{13}	(15, 13.5, 1.6)	T_{Jam}	T_6	0	8	[55, 70]	4
	T_{14}	(18, 3, 1.1)	T_{Dec}	T_{12}	0	7	/	3
	T_{15}	(18, 11.5, 2.4)	T_{Com}	T_4	0	13	/	5
	T_{16}	(17.5, 16.3, 2.7)	T_{Rec}	/	0	7	[40, 60]	7
	T_{17}	(15, 16.6, 2.7)	T_{Str}	/	1	4	/	2
	T_{18}	(18.5, 17, 3.5)	T_{Jam}	T_5	0	8	/	6

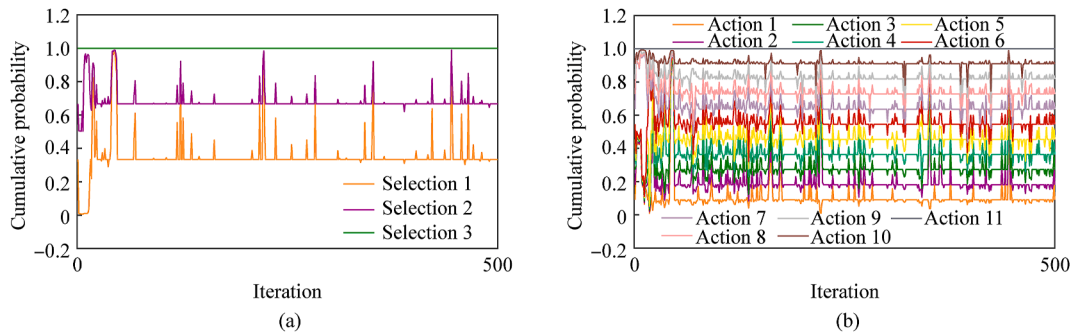


Fig. 8. The cumulative probability change curves of the operators in simple scene: (a) Selection operators; (b) Action operators.

represent the initial positions of the UAVs, while the green triangles indicate the terminal positions of the UAVs. From Fig. (10), it can be observed that 5 UAVs can complete 10 tasks. These results further demonstrate the effectiveness of the EL-HH algorithm in a simple scene.

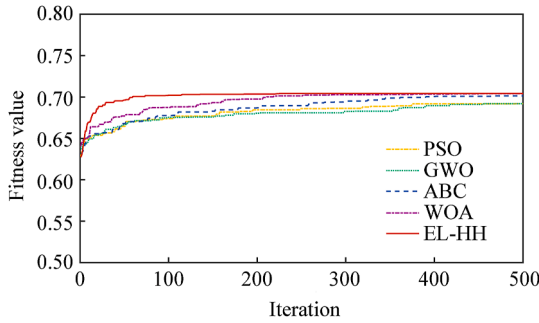
4.1.2. Complex scene

A complex scene with a flight space of $20 \text{ km} \times 20 \text{ km} \times 5 \text{ km}$ is created. In this scene, 6 UAVs have to complete 18 tasks. The UAV information is presented in Table 2, and the task information is shown in Table 3. The maximum iterations $T_{\max}$ is limited to 2500,

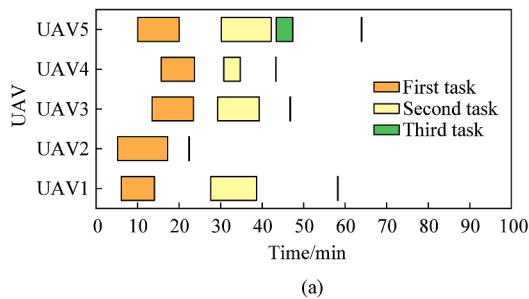
Table 4

The optimal results planned by all algorithms in simple scene.

Algorithm	UAV	Time/min	Task order (waiting time, time interval)
PSO	U_1	72.16	T_1 (0, [6.07, 14.07]), T_{10} (0, [27.62, 38.62]), T_6 (0, [51.32, 61.32])
	U_2	43.67	T_9 (0, [16.88, 26.88])
	U_3	25.26	T_5 (0, [7.82, 17.82])
	U_4	43.26	T_8 (0, [15.73, 23.73]), T_7 (0, [30.75, 34.75])
	U_5	69.01	T_3 (0, [15.63, 27.63]), T_2 (0, [35.27, 47.27]), T_4 (0, [48.51, 52.51])
GWO	U_1	58.20	T_1 (0, [6.07, 14.07]), T_{10} (0, [27.62, 38.62])
	U_2	34.71	T_2 (0, [11.27, 23.27])
	U_3	46.78	T_5 (0, [7.82, 17.82]), T_6 (0, [23.65, 33.65])
	U_4	64.67	T_8 (0, [15.73, 23.73]), T_4 (7.47, [40, 44]), T_7 (0, [52.15, 56.15])
	U_5	59.72	T_9 (0, [10.03, 20.03]), T_3 (0, [31.70, 43.70])
ABC	U_1	58.20	T_1 (0, [6.07, 14.07]), T_{10} (0, [27.62, 38.62])
	U_2	22.37	T_3 (0, [5.24, 17.24])
	U_3	46.75	T_6 (0, [13.47, 23.47]), T_5 (0, [29.30, 39.30])
	U_4	43.29	T_7 (0, [8.76, 12.76]), T_8 (0, [19.78, 27.78])
	U_5	63.89	T_9 (0, [10.03, 20.03]), T_2 (0, [30.16, 42.16]), T_4 (0, [43.39, 47.39])
WOA	U_1	58.20	T_1 (0, [6.07, 14.07]), T_{10} (0, [27.62, 38.62])
	U_2	22.37	T_3 (0, [5.24, 17.24])
	U_3	46.75	T_6 (0, [13.47, 23.47]), T_5 (0, [29.30, 39.30])
	U_4	43.29	T_7 (0, [8.76, 12.76]), T_8 (0, [19.78, 27.78])
	U_5	63.89	T_9 (0, [10.03, 20.03]), T_2 (0, [30.16, 42.16]), T_4 (0, [43.39, 47.39])
EL-HH	U_1	58.20	T_1 (0, [6.07, 14.07]), T_{10} (0, [27.62, 38.62])
	U_2	22.37	T_3 (0, [5.24, 17.24])
	U_3	46.75	T_6 (0, [13.47, 23.47]), T_5 (0, [29.30, 39.30])
	U_4	43.26	T_8 (0, [15.73, 23.73]), T_7 (0, [30.75, 34.75])
	U_5	63.89	T_9 (0, [10.03, 20.03]), T_2 (0, [30.16, 42.16]), T_4 (0, [43.39, 47.39])

**Fig. 9.** The comparative iterative change curves of average fitness in simple scene.

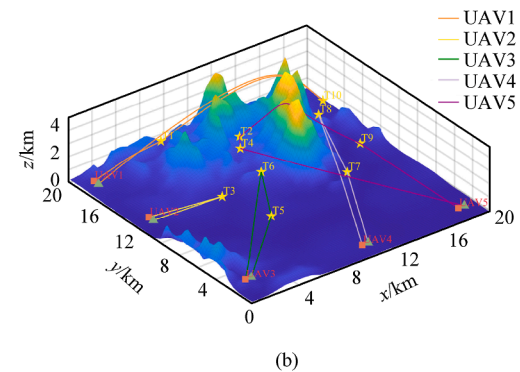
and the population size M is defined as 50. When the algorithm is run once for cooperative task assignment, the cumulative probability change curves for selection operators are shown in Fig. 11(a), and the cumulative probability change curves for action operators are shown in Fig. 11(b). These figures effectively validate that the energy learning hyper-heuristic strategy can efficiently adjust the selection probabilities of multiple low-level operators.



The optimal results achieved by all algorithms over multiple runs are listed in Table 5. The average cost variation curves over multiple runs are shown in Fig. 12. As illustrated in Fig. 12, the EL-HH algorithm can converge to the optimal scheme at the fastest speed. Since the algorithm incorporates the EL-HH strategy, which continuously selects the suitable optimization operators from multiple groups during the iterative process, it achieves the fastest optimization speed. Fig. 13(a) provides a chronological view, highlighting the order and duration of tasks executed by the UAVs. Fig. 13(b) visually maps out the flight paths, offering a clear representation within the map. From Fig. 13, it can be observed that 6 UAVs complete 18 tasks. These visual results further demonstrate the effectiveness of the EL-HH algorithm in a complex scene.

4.2. Experiment

The experimental setup consists of a 5 m × 5 m × 3 m indoor environment equipped with a motion capture system. Within this environment, several obstacles are placed, and their positions are known in advance. The UAVs must navigate around these obstacles during flight. The hardware platform consists of a planning workstation, an image processing workstation, a network switch, a

**Fig. 10.** The optimal result planned by the EL-HH algorithm in simple scene: (a) Chronological view; (b) Flight paths.

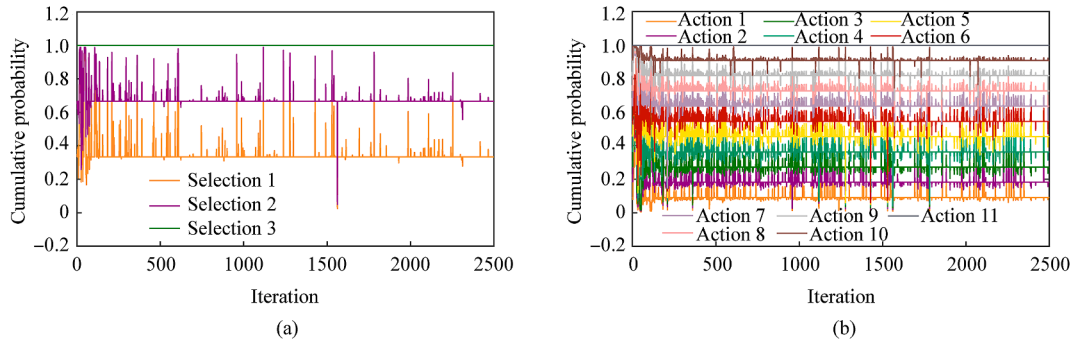


Fig. 11. The cumulative probability change curves of the operators in complex scene: (a) Selection operators; (b) Action operators.

Table 5

The optimal results planned by all algorithms in complex scene.

Algorithm	UAV	Time/min	Task order (waiting time, time interval)
PSO	U_1	39.34	T_1 (0.24, [5.00, 13.00]), T_8 (0, [23.07, 27.07])
	U_2	67.34	T_3 (0, [10.80, 21.80]), T_{15} (0, [35.95, 48.95])
	U_3	70.32	T_5 (0, [6.57, 13.57]), T_6 (0, [18.77, 24.77]), T_{12} (0, [30.00, 38.00]), T_{10} (0, [41.53, 54.53])
	U_4	80.34	T_{18} (0, [21.15, 29.15]), T_{17} (0, [32.78, 36.78]), T_7 (0, [44.50, 50.50]), T_{13} (0, [56.39, 64.39])
	U_5	57.16	T_{11} (0, [8.35, 15.35]), T_4 (0, [25.83, 29.83]), T_9 (0, [35.12, 41.12])
	U_6	78.53	T_2 (0, [20.01, 32.01]), T_{16} (0, [45.90, 52.90]), T_{14} (0, [66.50, 73.50])
GWO	U_1	36.65	T_1 (0.24, [5.00, 13.00]), T_5 (0, [23.07, 30.07])
	U_2	55.13	T_3 (0, [10.80, 21.80]), T_9 (0, [26.69, 32.69]), T_8 (0, [36.15, 40.15])
	U_3	81.83	T_6 (0, [11.57, 17.57]), T_{16} (11.78, [40.00, 47.00]), T_{10} (0, [53.04, 66.04])
	U_4	78.96	T_{18} (0, [21.15, 29.15]), T_{13} (20.52, [55.00, 63.00])
	U_5	70.30	T_7 (0, [10.61, 16.61]), T_4 (0, [19.89, 23.89]), T_{17} (0, [33.96, 37.96]), T_{15} (0, [44.14, 57.14])
	U_6	91.28	T_{12} (0, [8.58, 16.58]), T_{11} (0, [19.01, 26.01]), T_{14} (0, [30.88, 37.88]), T_2 (0, [58.86, 70.86])
ABC	U_1	38.38	T_1 (0.24, [5.00, 13.00]), T_2 (0, [17.22, 29.22])
	U_2	66.35	T_{15} (0, [18.21, 31.21]), T_{18} (0, [36.92, 44.92])
	U_3	45.63	T_{12} (0, [15.34, 23.34]), T_5 (0, [32.45, 39.45])
	U_4	83.76	T_3 (0, [14.63, 25.63]), T_4 (0, [29.28, 33.28]), T_8 (0, [39.65, 43.65]), T_7 (0, [47.91, 53.91]), T_{13} (0, [59.80, 67.80])
	U_5	56.64	T_6 (0.70, [10.00, 16.00]), T_9 (0, [23.15, 29.15]), T_{17} (0, [36.15, 40.15])
	U_6	77.68	T_{11} (0, [6.53, 13.53]), T_{14} (0, [18.40, 25.40]), T_{16} (1.01, [40.00, 47.00]), T_{10} (0, [53.04, 66.04])
WOA	U_1	38.38	T_1 (0.24, [5.00, 13.00]), T_2 (0, [17.22, 29.22])
	U_2	33.05	T_3 (0, [10.80, 21.80])
	U_3	45.79	T_5 (0, [6.57, 13.57]), T_{11} (0, [23.49, 30.49])
	U_4	78.96	T_7 (0, [11.53, 17.53]), T_8 (0, [21.79, 25.79]), T_{17} (0, [29.83, 33.83]), T_{18} (0, [37.46, 45.46]), T_{13} (4.21, [55.00, 63.00])
	U_5	63.95	T_4 (0, [12.10, 16.10]), T_9 (0, [21.39, 27.39]), T_{15} (0, [37.78, 50.78])
	U_6	84.74	T_6 (0, [10.32, 16.32]), T_{12} (0, [21.54, 29.54]), T_{10} (0, [33.07, 46.07]), T_{16} (0, [52.11, 59.11]), T_{14} (0, [72.70, 79.70])
EL-HH	U_1	38.38	T_1 (0.24, [5.00, 13.00]), T_2 (0, [17.22, 29.22])
	U_2	33.05	T_3 (0, [10.80, 21.80]), T_9 (0, [26.69, 32.69]), T_8 (0, [36.15, 40.15])
	U_3	35.95	T_6 (0, [11.57, 17.57]), T_5 (0, [22.77, 29.77])
	U_4	78.96	T_4 (0, [11.73, 15.73]), T_{17} (0, [25.81, 29.81]), T_{18} (0, [33.43, 41.43]), T_{13} (8.24, [55.00, 63.00])
	U_5	63.95	T_7 (0, [10.61, 16.61]), T_{15} (0, [25.26, 38.26])
	U_6	82.48	T_{12} (0, [8.58, 16.58]), T_{10} (0, [20.11, 33.11]), T_{16} (0.85, [40.00, 47.00]), T_{11} (0, [56.70, 63.70]), T_{14} (0, [68.57, 75.57])

WiFi device, multiple motion capture cameras, and several quadrotor UAVs. The planning workstation is a computer equipped with Ubuntu and MATLAB. The image workstation is installed with a motion capture system and is connected to multiple cameras via

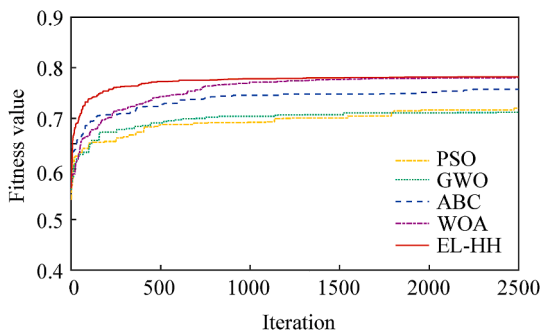


Fig. 12. The comparative iterative change curves of average fitness in complex scene.

the network switch. The motion capture system captures the reflective signals from the UAVs using the cameras, and converts them into image data. The image workstation processes image data into motion data and sends it to the planning workstation. Based on the planning scheme, the planning workstation sends control commands to the UAVs via the WiFi device. An illustration of the experimental environment is shown in Fig. 14. The task assignment problem involves 3 UAVs cooperating to complete 6 tasks. The details of the problem are provided in Tables 6 and 7. The EL-HH algorithm is applied for cooperative task assignment, with the number of iterations set to 500.

The assignment scheme is shown in Table 8. Fig. 15(a) provides a chronological view, highlighting the order and duration of tasks executed by the UAVs. Fig. 15(b) visually maps out the flight paths, offering a clear representation within the map. Subsequently, 3 UAVs execute the assignment scheme, and the flight process is illustrated in Fig. 16. Fig. 16 visually maps the real flight paths and task positions, providing a clear representation of the execution. The UAVs not only avoid obstacles but also complete the assigned

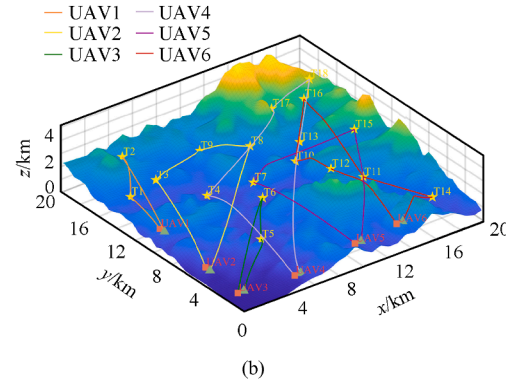
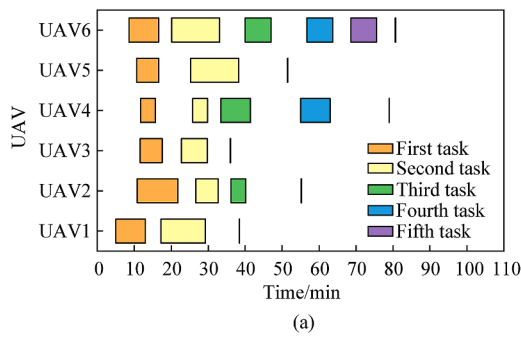


Fig. 13. The optimal result planned by the EL-HH algorithm in complex scene: (a) Chronological view; (b) Flight paths.

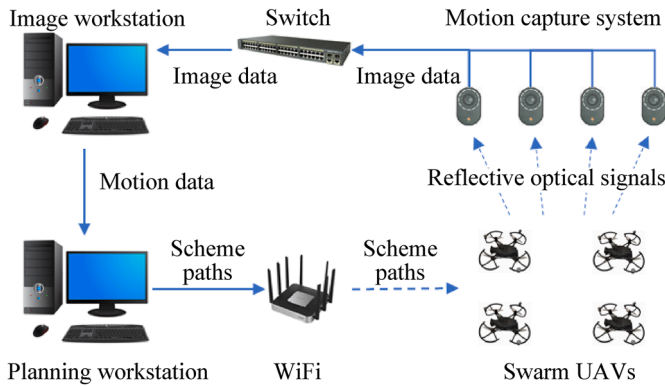


Fig. 14. The experimental environment.

tasks, confirming the algorithm's capability to obtain the best scheme in a real environment.

5. Conclusions

This paper proposes the EL-HH algorithm to address the cooperative task assignment problem of heterogeneous UAVs under complex constraints. First, a mathematical model is developed to define the problem scenario, complex constraints, and objective function. Then, the scheme encoding, the EL-HH strategy, multiple optimization operators, and the task sequence and time

Table 8

The optimal result in real environment.

Algorithm	UAV	Time/min	Task order (waiting time, time interval)
EL-HH	U_1	2.95	T_5 (0, [0.77, 1.77]), T_1 (0, [2.36, 2.66])
	U_2	2.29	T_2 (0, [0.72, 1.02]), T_4 (0, [1.22, 1.52])
	U_3	2.08	T_6 (0, [0.72, 1.02]), T_3 (0, [1.43, 1.73])

adjustment strategy are designed in the EL-HH algorithm. Through these designs, the EL-HH algorithm efficiently explores the solution space while accelerating the search for the optimal solution. The simulation and experiment demonstrate the effectiveness of the proposed algorithm in solving the cooperative task assignment problem. The EL-HH algorithm significantly enhances the efficiency and reliability of cooperative task assignment of heterogeneous UAVs, providing a robust method for complex scenarios. The focus of future work is to further improve the hyper-heuristic strategy and enhance the algorithm's efficiency.

CRediT authorship contribution statement

Mengshun Yuan: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Mou Chen:** Writing – review & editing, Supervision, Project administration, Funding acquisition, Formal analysis, Conceptualization. **Tongle Zhou:** Visualization, Validation, Methodology, Investigation, Data curation. **Zengliang Han:** Validation, Software, Investigation, Formal analysis.

Table 6

UAV information in real environment.

Scene	UAV	Initial position/km	Terminal position/km	Load type	Number of Missiles	Maximum time
Real	U_1	(1.3, 0.3, 0.5)	(1.1, 0.3, 0.5)	L_{Rec}, L_{Dec}	0	70
	U_2	(2.5, 0.3, 0.5)	(2.3, 0.3, 0.5)	L_{Jam}, L_{Dec}	0	40
	U_3	(3.7, 0.3, 0.5)	(3.5, 0.3, 0.5)	L_{Rec}, L_{Str}	0	50

Table 7

Task information in real environment.

Scene	UAV	Position/km	Task type	Prior Task	Missile quantity	Execution/min	Window/min	Value
Real	T_1	(0.6, 1.9, 0.7)	T_{Rec}	T_4	0	0.3	/	6
	T_2	(1.9, 4.5, 0.7)	T_{Jam}	/	0	0.3	/	2
	T_3	(4.5, 1.9, 0.7)	T_{Rec}	T_2	0	0.3	[1.3, 2]	4
	T_4	(0.7, 4.3, 0.7)	T_{Dec}	/	1	0.3	/	3
	T_5	(3.1, 4.2, 0.7)	T_{Com}	/	0	1	[0.5, 2]	5
	T_6	(4.3, 4.3, 0.7)	T_{Str}	/	1	0.3	/	1

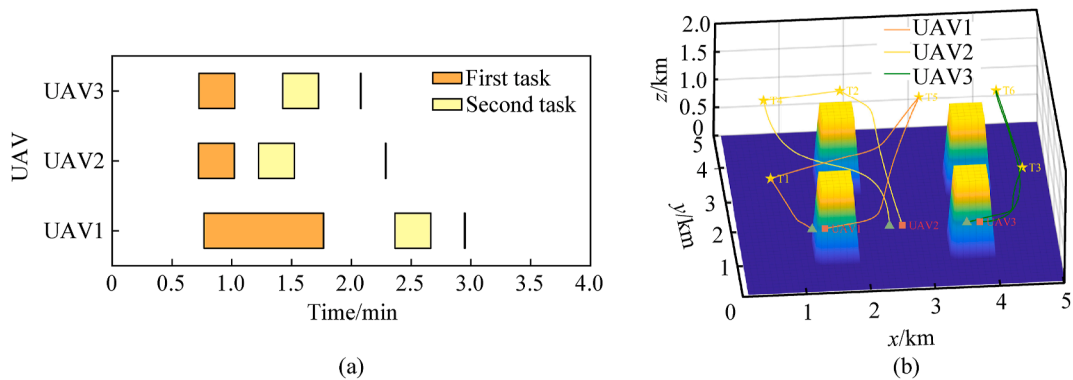


Fig. 15. The assignment result in real environment: (a) Chronological view; (b) Flight paths.

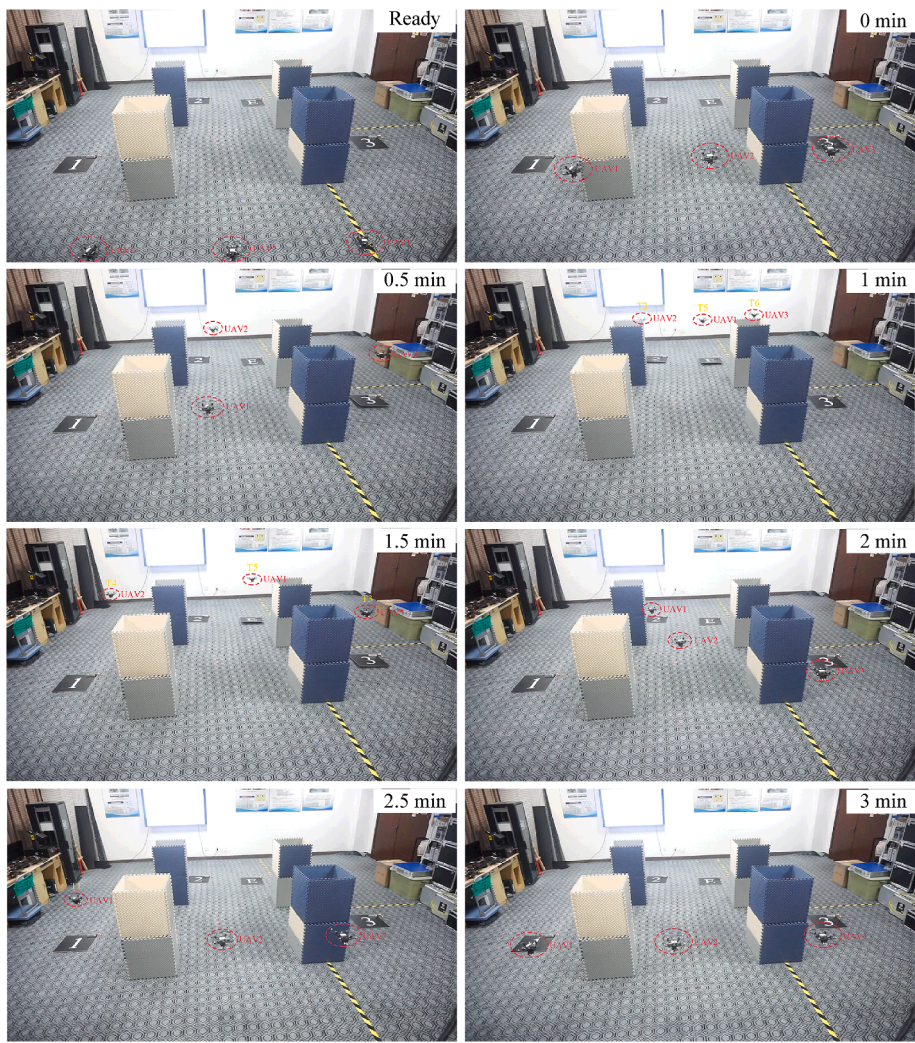


Fig. 16. The flight process in real environment.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was funded by the National Natural Science Foundation of China (Grant No. 62203217), the Jiangsu Province Basic Research Program Natural Science Foundation (Grant No. BK20220885), the Hong Kong, Macao and Taiwan Science and Technology Cooperation Project of Special Foundation in Jiangsu

Science and Technology Plan (Grant No. BZ2023057), the Fundamental Research Funds for the Central Universities (Grant No. NJ2024012), the China Postdoctoral Science Foundation (Grant No. GZC20242230), and the Postgraduate Research & Practice Innovation Program of Jiangsu Province (Grant No. KYCX24_0586).

References

- [1] Zheng R, Lyu Y. Nonlinear tight formation control of multiple UAVs based on model predictive control. *Def Technol* 2023;25:69–75.
- [2] Zhou X, Wen XY, Wang ZP, Gao YM, Li HJ, Wang QH, et al. Swarm of micro flying robots in the wild. *Sci Robot* 2022;7:eabm5954.
- [3] Zuo L, Gao S, Li YC, Li LH, Li M, Lu XF. A fast and robust algorithm with reinforcement learning for large UAV cluster mission planning. *Remote Sens* 2022;14:1304.
- [4] Ma HX, Chen M, Feng G, Wu QX. Disturbance-observer-based adaptive fuzzy tracking control for unmanned autonomous helicopter with flight boundary constraints. *IEEE Trans Fuzzy Syst* 2023;31:184–98.
- [5] Yuan MS, Zhou TL, Chen M. Improved lazy theta* algorithm based on octree map for path planning of UAV. *Def Technol* 2023;23:8–18.
- [6] Wang C, Huo X, Ma KM, Ji RH. PID-like model free adaptive control with discrete extended state observer and its application on an unmanned helicopter. *IEEE Trans Ind Informat*, vol. 19; 2023. p. 11265–74.
- [7] Wang JN, Wang DD, Wang CY, Deng F. Robust formation control for unmanned helicopters with collision avoidance. *J Franklin Inst* 2020;357:11997–2018.
- [8] Gu X, Xian B, Wang YX. Geometry-based adaptive tracking control for an underactuated small-size unmanned helicopter. *IEEE Trans Syst, Man, Cybern, Syst* 2023;53:489–500.
- [9] Duan DY, Zu R, Yu TL, Zhang CQ, Li JB. Differential flatness-based real-time trajectory planning for multi-helicopter cooperative transportation in crowded environments. *AIAA J* 2023;1:79–95.
- [10] Han ZL, Chen M, Shao SY, Wu QX. Improved artificial bee colony algorithm-based path planning of unmanned autonomous helicopter using multi-strategy evolutionary learning. *Aero Sci Technol* 2022;22:07374.
- [11] Li K, Yan XX, Han Y. Multi-mechanism swarm optimization for multi-UAV task assignment and path planning in transmission line inspection under multi-wind field. *Appl Soft Comput* 2024;50:11033.
- [12] Zhang C, Luo XQ, Liang JW, Liu XM, Zhu LH, Guo S. POTA: privacy-preserving online multi-task assignment with path planning. *IEEE Trans Mobile Comput* 2024;3:999–6011.
- [13] Zhang TX, Wang YH, Ma ZJ, Kong LJ. Task assignment in UAV-enabled front jammer swarm: a coalition formation game approach. *IEEE Trans Aero Electron Syst* 2023;9:562–75.
- [14] Gao XH, Wang L, Yu XY, Su XC, Ding Y, Lu C, et al. Conditional probability based multi-objective cooperative task assignment for heterogeneous UAVs. *Eng Appl Artif Intell* 2023;23:06404.
- [15] Zheng D, Zhang Y, Li F, Cheng P. UAVs cooperative task assignment and trajectory optimization with safety and time constraints. *Def Technol* 2023;0:49–61.
- [16] Poudel S, Moh S. Task assignment algorithms for unmanned aerial vehicle networks: a comprehensive survey. *Veh Commun* 2022;5:00469.
- [17] Wang GH, Wang FM, Wang JH, Li MZ, Gai L, Xu DC. Collaborative target assignment problem for large-scale UAV swarm based on two-stage greedy auction algorithm. *Aero Sci Technol* 2024;49:09146.
- [18] Schuster R, Hanson JO, Strimas-Mackey M, Bennett JR. Exact integer linear programming solvers outperform simulated annealing for solving conservation planning problems. *PeerJ* 2020;8:e9258.
- [19] Wang J, Cevik M, Amin SH, Parsaee AA. Mixed-integer linear programming models for the paint waste management problem. *Transp Res Part E Logist Transp Rev* 2021;151:102343.
- [20] Thibbotuwawa A, Bociewicz G, Radzki G, Nielsen P, Banaszak Z. UAV mission planning resistant to weather uncertainty. *Sens* 2020;20:515.
- [21] Chang ZX, Chen YN, Yang WY, Zhou ZB. Mission planning problem for optical video satellite imaging with variable image duration: a greedy algorithm based on heuristic knowledge. *Adv Space Res* 2020;66:2597–609.
- [22] Zhang GH, Li XH, Hu GX, Zhang ZB, An JP, Man WX. Mission planning issues of imaging satellites: summary, discussion, and prospects. *Int J Aerosp Eng* 2021;2021:7819105.
- [23] Gao XQ, Liu RK, Kaushik A. Service chaining placement based on satellite mission planning in ground station networks. *IEEE Trans Netw Service Manag* 2021;18:3049–63.
- [24] Zhou Z, Li FM, Zhu HX, Xie HL, Abawajy JH, Chowdhury MU. An improved genetic algorithm using greedy strategy toward task scheduling optimization in cloud environments. *Neural Comput Appl* 2020;32:1531–41.
- [25] Lin LH, Wang ZG, Tian LQ, Wu JY, Wu WX. A PSO-based energy-efficient data collection optimization algorithm for UAV mission planning. *PLoS One* 2024;19:e0297066.
- [26] Yu XB, Jiang NJ, Wang XM, Li MY. A hybrid algorithm based on grey wolf optimizer and differential evolution for UAV path planning. *Expert Syst Appl* 2023;215:119327.
- [27] Han ZL, Chen M, Shao SY, Zhu HJ, Wu QX. Cooperative multi-task assignment of unmanned autonomous helicopters based on hybrid enhanced learning ABC algorithm. *IEEE Trans Intell Veh* 2024;9:526–40.
- [28] Dai YN, Yu JY, Zhang C, Zhan BW, Zheng XT. A novel whale optimization algorithm of path planning strategy for mobile robots. *Appl Intell* 2023;53:10843–57.
- [29] Saif FA, Latip R, Hanapi ZM, Shafinah K. Multi-objective grey wolf optimizer algorithm for task scheduling in cloud-fog computing. *IEEE Access* 2023;11:20635–46.
- [30] Wang X, Wu J, Shi Z, Zhao FY, Jin ZH. Deep reinforcement learning-based autonomous mission planning method for high and low orbit multiple agile earth observing satellites. *Adv Space Res* 2022;70:3478–93.
- [31] Yue LF, Yang RN, Zhang Y, Yu LX, Wang ZZ. Deep reinforcement learning for UAV intelligent mission planning. *Complexity* 2022;2022:3551508.
- [32] Luo D, Fan Z, Yang Z, Xu Y. Multi-UAV cooperative maneuver decision-making for pursuit-evasion using improved MADRL. *Def Technol* 2024;35:187–97.
- [33] Chen SM, Li ZY, Yang B, Rudolph G. Quantum-inspired hyper-heuristics for energy-aware scheduling on heterogeneous computing systems. *IEEE Trans Parallel Distr Syst* 2016;27:1796–810.
- [34] Zhang S, Xu YB, Zhang WY. Multitask-oriented manufacturing service composition in an uncertain environment using a hyper-heuristic algorithm. *J Manuf Syst* 2021;60:138–51.
- [35] Yan FH, Di K. Solving the multi-robot task allocation with functional tasks based on a hyper-heuristic algorithm. *Appl Soft Comput* 2023;146:110628.
- [36] Shao ZS, Shao WS, Pi DC. LS-HH: a learning-based selection hyper-heuristic for distributed heterogeneous hybrid blocking flow-shop scheduling. *IEEE Trans Emerg Topics Comput Intell* 2023;7:111–27.
- [37] Zhao BC, Huo MY, Li Z, Yu Z, Qi NM. Clustering-based hyper-heuristic algorithm for multi-region coverage path planning of heterogeneous UAVs. *Neurocomputing* 2024;610:128528.
- [38] Jin XZ, Zhang GX, Yang M, Peng XC, Shi BH. Cooperative multi-task traversing with complex marine environment for multiple unmanned surface vehicles inspired by membrane computing. *Ocean Eng* 2022;266:112586.
- [39] Chen MR, Huang YY, Zeng GQ, Lu KD, Yang LQ. An improved bat algorithm hybridized with extremal optimization and Boltzmann selection. *Expert Syst Appl* 2021;175:114812.
- [40] Wang Y, Yin BS, Chen ZY, Tao MX. Time-efficient ring-topology design for decentralized learning via device-to-device communications. *IEEE Wireless Commun Lett* 2024;13:3192–6.
- [41] Yang L, Zhong CY, Yang QH, Zou WR, Fathalla A. Task offloading for directed acyclic graph applications based on edge computing in industrial internet. *Inf Sci* 2020;540:51–68.